EXPERIMENT – 1

1. To implement simple search method using MoveGen(N) and GoalTest(N) where N is concerned node.

Apparatus: Python, NumPy

Theory: Most problem-solving techniques in Artificial Intelligence are goal-oriented, designed to transform an initial state into a desired outcome. These problem spaces are typically represented using graphs or state-space decision trees. Before exploring specific graph traversal strategies like Depth-First Search (DFS) or Breadth-First Search (BFS), it is essential to understand the fundamental Simple Search Algorithm. This baseline approach operates on the core principle of Generate-and-Test:

i. **Generate**: Dynamically generate successor nodes (children) from the current node (parent) using a move generator function (MoveGen()).
ii. **Test**: Apply a goal test function to validate whether a generated node matches the target outcome.

To maintain state and manage the search process, the algorithm tracks and manipulates nodes using the following parameters and operations:

i. **Start**: The initial state or root node where the search originates.
ii. **Goal**: The targeted state used to evaluate whether the search has succeeded.
iii. **OPEN**: A list that stores all generated nodes that are pending expansion or evaluation.
iv. **CLOSED**: A list that keeps track of all nodes that have already been expanded or visited, preventing redundant processing and infinite loops.
v. **MoveGen**: The operator that produces all valid child nodes for a given node.
vi. **Append**: Inserts newly generated children into the OPEN list.
vii. **Sort**: Reorders the OPEN list based on predefined criteria or heuristics.

Code:

```
import numpy as np

class Node:
    def __init__(self, value: str):
        self.value = value
        self.neighbors: list[Node] = []
    def __repr__(self):
        return self.value

def move_gen(node: Node) -> list[Node]:
    return node.neighbors

def goal_test(node: Node, goal: Node) -> bool:
    return node == goal

def simple_search(start: Node, goal: Node) -> tuple[bool, list[Node]]:
    open = np.array([start], dtype=object)
    closed = np.array([], dtype=object)

    while len(open) > 0:
```

```
        node = open[0]
        open = open[1:]
        closed = np.append(closed, node)

        if goal_test(node, goal):
            return True, closed.tolist()

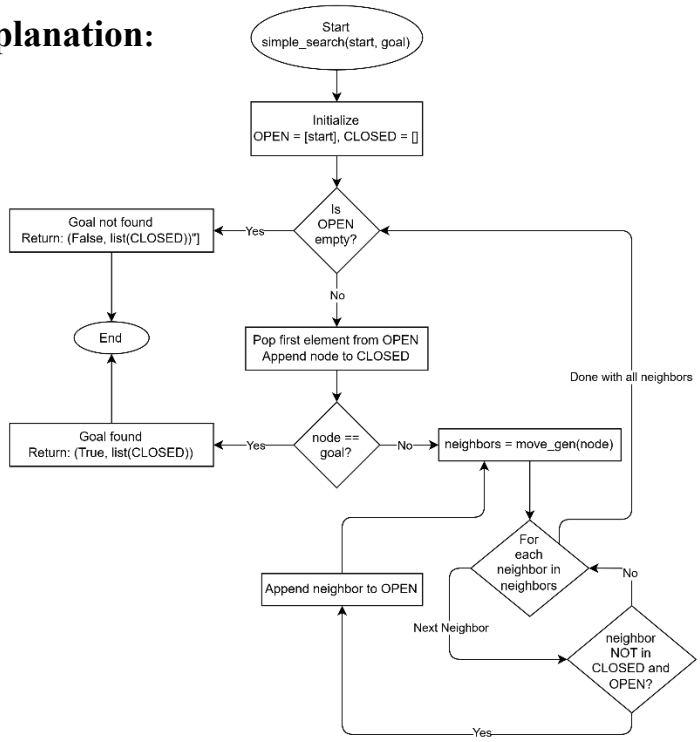
        neighbors = move_gen(node)
        for neighbor in neighbors:
            if neighbor not in closed and neighbor not in open:
                open = np.append(open, neighbor)

    return False, closed.tolist()

if __name__ == "__main__":
    a = Node("A")
    b = Node("B")
    c = Node("C")
    d = Node("D")
    a.neighbors.extend([b, c])
    b.neighbors.extend([a, c, d])
    c.neighbors.extend([a, b, d])
    d.neighbors.extend([c, b])

    status, path = simple_search(a, d)
    print("Search Result:")
    print("\tStart :", a)
    print("\tGoal :", d)
    print("\tGoal Found :", status)
    print("\tVisited:", " -> ".join(str(node) for node in path))
```

Code Explanation:



Output:

```
Search Result:
        Start : A
        Goal : D
        Goal Found : True
        Visited: A -> B -> C -> D
```

EXPERIMENT – 2

2.1: To implement Breadth First Search algorithm using Python.

Apparatus: Python

Theory: Breadth-First Search (BFS) is a graph traversal algorithm used to explore the nodes of a graph level by level. It starts from a given node and visits all its adjacent nodes before moving to the nodes at the next level.

In the given implementation, the graph is represented using a **dictionary**, where each key represents a node and its corresponding value contains a list of neighboring nodes. BFS uses the following data structures:

i. Graph: The graph is stored using a Python dictionary. Each node is associated with a list of its directly connected neighboring nodes.

ii. Queue: A deque from Python's collections module is used as the queue. It follows the **FIFO (First In, First Out)** principle. The popleft() operation removes the node at the front of the queue, while newly discovered nodes are added to the rear using append().

iii. Visited: A Python **set** is used to store nodes that have already been discovered. It prevents the same node from being added to the queue multiple times and avoids unnecessary repeated traversal.

iv. Order: A Python **list** is used to maintain the order in which nodes are removed from the queue and visited during the search.

v. Start Node: The node from which the BFS traversal begins. In this program, the starting node is A.

vi. Goal Node: The node that the search attempts to find. In this program, goal node is D.

The BFS process begins by placing the start node into the queue and marking it as visited. The node at the front of the queue is then removed and added to the visited order. If it is not the goal, its unvisited neighboring nodes are added to the queue and marked as visited. This process continues until the goal node is found or the queue becomes empty. The use of a queue ensures that nodes are explored level by level, while the visited set prevents repeated processing of nodes.

Code:

```
from collections import deque

GRAPH = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}


def breadth_first_search(start: str, goal: str) -> tuple[bool, list[str]]:
    queue = deque([start])
    visited = {start}
    order = []

    while queue:
        node = queue.popleft()
```

```
        order.append(node)

        if node == goal:
            return True, order

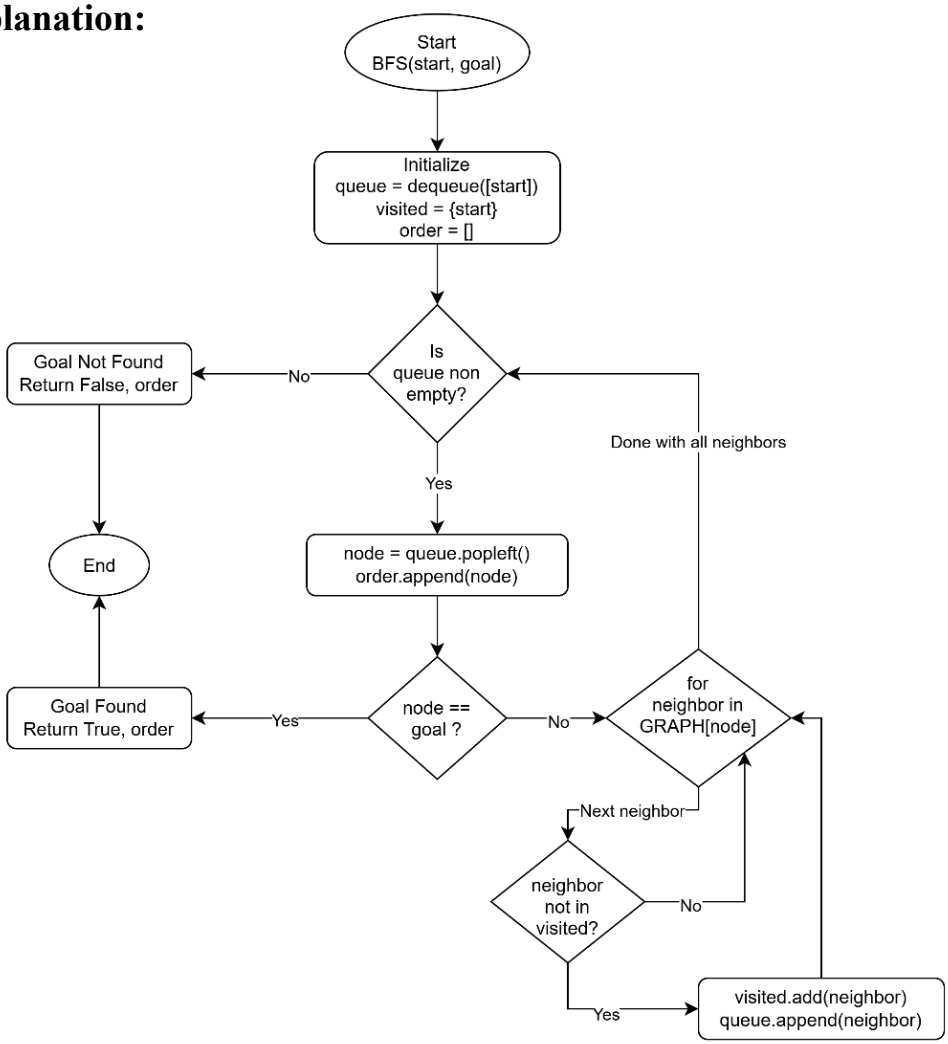
        for neighbor in GRAPH[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return False, order


if __name__ == "__main__":
    status, path = breadth_first_search("A", "D")

    print("Search Result:")
    print("\tStart : A")
    print("\tGoal : D")
    print("\tGoal Found :", status)
    print("\tVisited:", " -> ".join(path))
```

Code Explanation:



Output:

```
Search Result:
        Start : A
        Goal : D
        Goal Found : True
        Visited: A -> B -> C -> D
```

2.2: To implement Depth First Search algorithm using Python.

Apparatus: Python

Theory: Depth-First Search (DFS) is a graph traversal algorithm used to explore the nodes of a graph by going as deep as possible along each branch before backtracking. It starts from a given node and explores an unvisited neighboring node before moving to other neighboring nodes. In the given implementation, the graph is represented using a **dictionary**, where each key represents a node and its corresponding value contains a list of neighboring nodes. DFS uses the following data structures:

i. Graph: The graph is stored using a Python **dictionary**. Each node is associated with a list of its directly connected neighboring nodes.

ii. Stack: A Python **list** is used as a stack. It follows the **LIFO (Last In, First Out)** principle. The pop() operation removes the node from the top of the stack, while newly discovered nodes are added using append().

iii. Visited: A Python **set** is used to store nodes that have already been discovered. It prevents the same node from being added to the stack multiple times and avoids unnecessary repeated traversal.

iv. Order: A Python **list** is used to maintain the order in which nodes are removed from the stack and visited during the search.

v. Start Node: The node from which the DFS traversal begins. In this program, the starting node is **A**.

vi. Goal Node: The node that the search attempts to find. In this program, goal node is **D**.

The DFS process begins by placing the start node into the stack and marking it as visited. The node at the top of the stack is then removed and added to the visited order. If it is not the goal node, its unvisited neighboring nodes are added to the stack and marked as visited. The process continues until the goal node is found or the stack becomes empty. The use of a stack ensures that the search explores one branch as deeply as possible before backtracking, while the visited set prevents repeated processing of nodes.

Code:

```
GRAPH = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

def depth_first_search(start: str, goal: str) -> tuple[bool, list[str]]:
    stack = [start]
    visited = {start}
    order = []

    while stack:
        node = stack.pop()
        order.append(node)

        if node == goal:
            return True, order
```

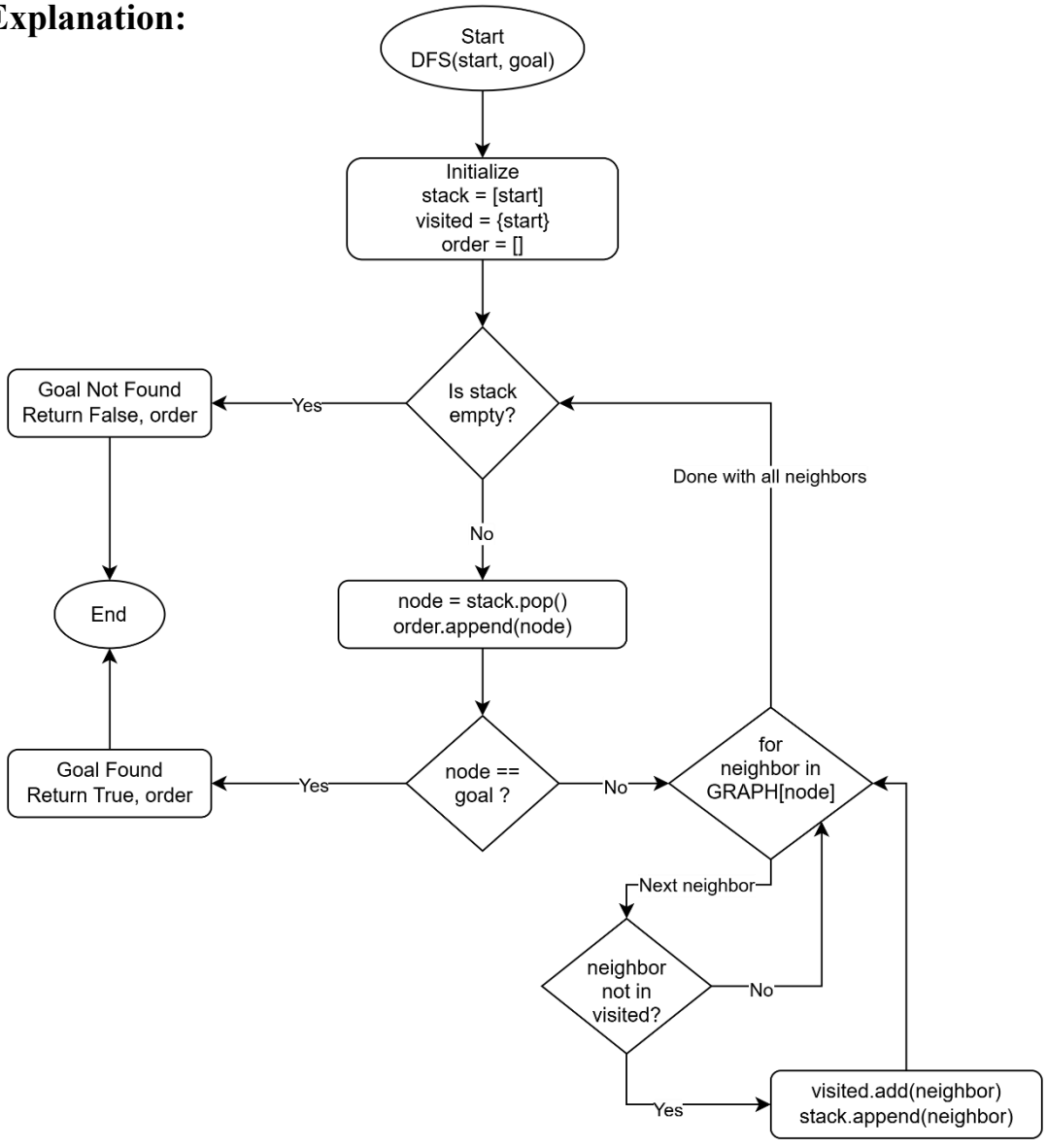
```
        for neighbor in GRAPH[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append(neighbor)

    return False, order


if __name__ == "__main__":
    status, path = depth_first_search("A", "D")

    print("Search Result:")
    print("\tStart : A")
    print("\tGoal : D")
    print("\tGoal Found :", status)
    print("\tVisited:", " -> ".join(path))
```

Code Explanation:



Output:

```
Search Result:
        Start : A
        Goal : D
        Goal Found : True
        Visited: A -> C -> D
```

EXPERIMENT – 3

3. To implement the Constraint Satisfaction Problem (CSP) using Python with the backtracking algorithm.

Apparatus: Python

Theory: A Constraint Satisfaction Problem (CSP) is a problem in which a set of variables must be assigned values from their respective domains while satisfying a set of constraints.
A CSP consists of three main components:

1. **Variables**: The objects that need to be assigned values. In this program, the variables are A, B, C, and D.
2. **Domains**: The possible values that can be assigned to each variable. In this program, every variable has the domain [1, 2, 3].
3. **Constraints**: The restrictions that determine which assignments are valid.

In this implementation, the graph is represented using a dictionary, where each key represents a node and its corresponding value contains a list of neighboring nodes. The graph defines the relationships between variables.
The following constraints are applied:

- $A \neq B$
- $B \neq C$
- $A \neq C$
- $B < D$
- $C \neq D$

The program uses the following data structures:

i. Graph: The graph is stored using a Python dictionary. Each node is associated with a list of its directly connected neighboring nodes.

ii. Domains: A Python dictionary is used to store the possible values for each variable. Every variable has the domain [1, 2, 3].

iii. Solution: A Python dictionary is used to store the final assignment of values to the variables.

iv. Assignment: A Python dictionary is used to maintain the values assigned to variables during the search process.

v. Backtracking: Backtracking is a systematic search technique that assigns values to variables one by one. If an assignment violates any constraint, it is rejected. If no valid value is available, the algorithm backtracks to the previous variable and tries another value.

vi. Constraint Checking: The is_valid() function checks whether assigning a particular value to a variable satisfies all the constraints with the variables that have already been assigned.

Working Principle: The CSP process begins by creating a list of variables from the domain dictionary. An empty assignment dictionary is initialized to store the values assigned during the search. The backtracking function selects one variable at a time and tries each value from its domain. Before assigning a value, the is_valid() function checks whether the assignment satisfies all the given constraints. If the value is valid, it is added to the assignment dictionary and the algorithm recursively proceeds to the next variable. If a complete assignment is obtained, the search is successful. If a constraint is violated at any stage, the algorithm removes the assigned value and tries another value. The process continues until a valid solution is found or all possible assignments have been explored.

Code:

```
GRAPH = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

DOMAINS = {
    "A": [1, 2, 3],
    "B": [1, 2, 3],
    "C": [1, 2, 3],
    "D": [1, 2, 3]
}

SOLUTION = {}

def constraint_satisfaction_problem() -> list[tuple[str, int]]:
    variables = list(DOMAINS)
    assignment = {}

    def is_valid(variable: str, value: int) -> bool:
        for neighbor in GRAPH[variable]:
            if neighbor not in assignment:
                continue

            if variable == "B" and neighbor == "D":
                if value >= assignment[neighbor]:
                    return False

            elif variable == "D" and neighbor == "B":
                if assignment[neighbor] >= value:
                    return False

            elif value == assignment[neighbor]:
                return False

        return True

    def backtrack(index: int) -> bool:
        if index == len(variables):
            return True

        variable = variables[index]

        for value in DOMAINS[variable]:
            if is_valid(variable, value):
                assignment[variable] = value

                if backtrack(index + 1):
```

```
                return True

            del assignment[variable]

        return False

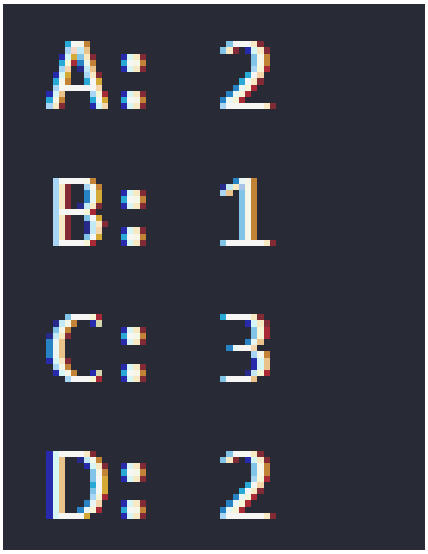
    if backtrack(0):
        SOLUTION.update(assignment)
        return list(SOLUTION.items())

    return []

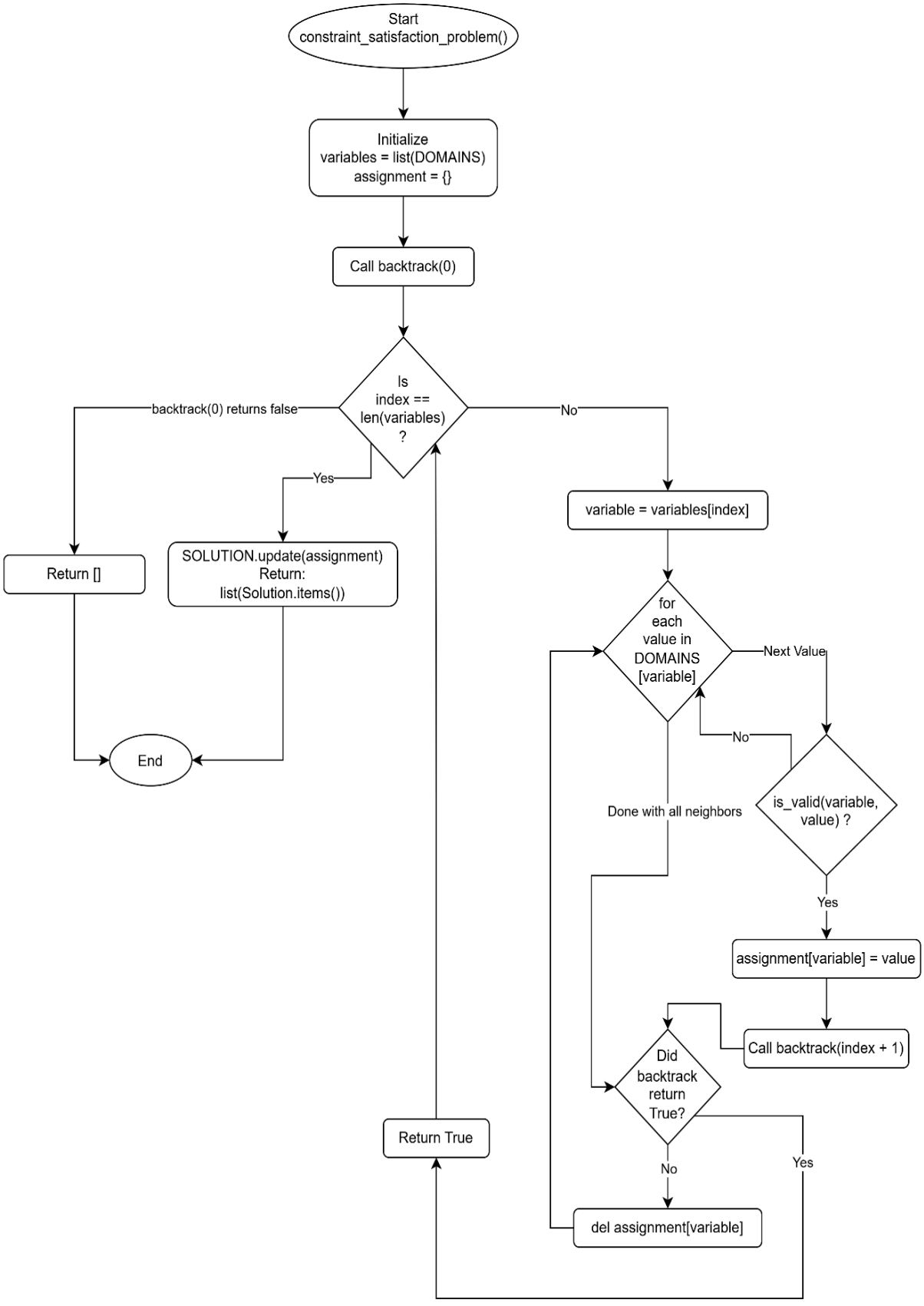
if __name__ == "__main__":
    solution = constraint_satisfaction_problem()

    for variable, value in solution:
        print(f"{variable}: {value}")
```

Output:

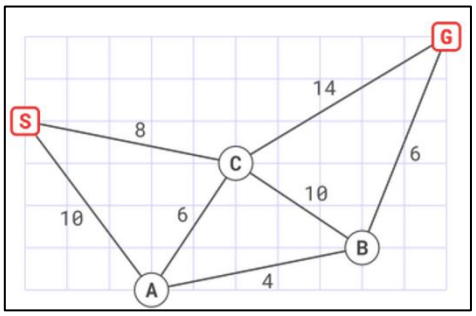


Code Explanation:



EXPERIMENT – 4

4. To implement the Hill Climbing search algorithm using Python to find a path from a start node to a goal node using Manhattan distance as the heuristic function.



Apparatus: Python

Theory: Hill Climbing is a heuristic search algorithm used in Artificial Intelligence to find a solution by continuously moving from the current state to a neighboring state that appears better according to a heuristic function. It is a local search algorithm because it considers the current state and its neighboring states rather than maintaining a complete search tree. In this implementation, the graph contains five nodes: S, A, C, B, and G. The objective is to find a path from the start node S to the goal node G.

Heuristic Function: The heuristic function used in this program is Manhattan distance. Manhattan distance calculates the distance between two points by adding the absolute differences of their x-coordinates and y-coordinates.

$$h(n) = | x_n - x_G | + | y_n - y_G |$$

where:

$h(n)$is the heuristic value of node n.
$(x_n, y_n)$are the coordinates of the current node.
$(x_G, y_G)$are the coordinates of the goal node G.

The graph is represented using a Python dictionary, where each key represents a node and its corresponding value contains a list of neighboring nodes. The graph is undirected, meaning movement is possible in both directions along the edges. The coordinates of the nodes are:

Node	Coordinates	Manhattan Distance
S	(0, 4)	12
A	(3, 0)	13
C	(5, 3)	8
B	(9, 1)	6
G	(11, 5)	0

Working Principle: The Hill Climbing process begins at the start node S. The algorithm examines all neighboring nodes and selects the neighbor having the lowest heuristic value. The selected neighbor is accepted only if its heuristic value is strictly smaller than the current node's heuristic value. If no neighbor provides an improvement, the algorithm stops because a local optimum has been reached.

The process continues until the goal node is reached or no better neighboring node is available.

Data Structures Used

i. Graph: A Python dictionary storing nodes and their neighboring nodes.
ii. Heuristic Dictionary: Stores the Manhattan distance of each node from the goal.
iii. Path: A Python list storing the nodes visited during the search.
iv. Current Node: Stores the node currently being explored.

Code:

```
def hill_climbing(graph, h, start, goal):
    current = start
    path = [current]

    print("=== STARTING HILL CLIMBING SEARCH (MANHATTAN) ===")

    while current != goal:
        neighbors = graph[current]

        if not neighbors:
            print("No neighbors found. Search failed.")
            return path

        best_neighbor = min(neighbors, key=lambda node: h[node])

        print(
            f"Current Node: {current} (h={h[current]}) "
            f"--> Best Neighbor: {best_neighbor} (h={h[best_neighbor]})"
        )

        if h[best_neighbor] >= h[current]:
            print("Local optimum reached. Halting search.")
            break

        current = best_neighbor
        path.append(current)

    if current == goal:
        print("Success! Goal reached.")
    else:
        print("Failed to reach the goal.")

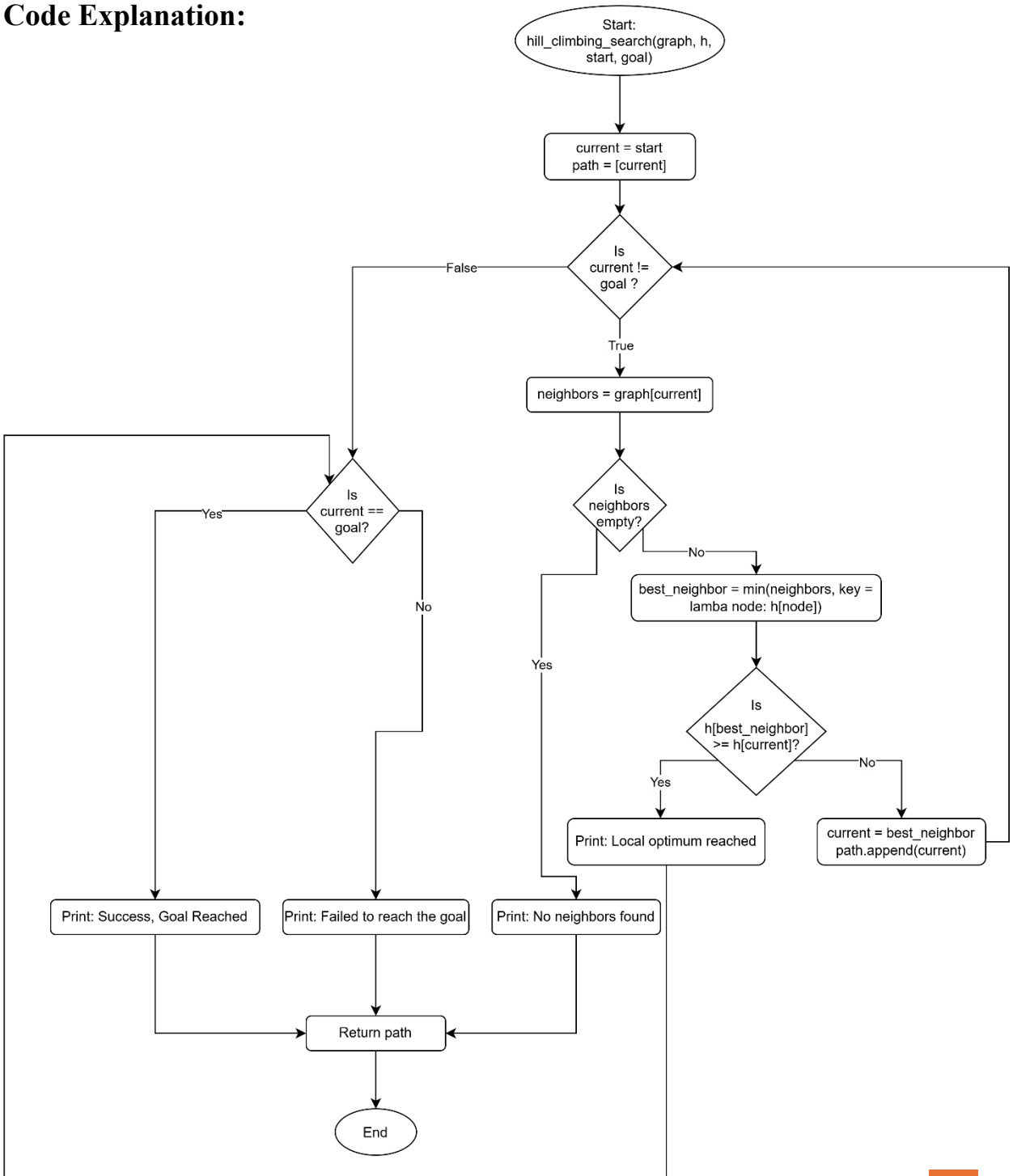
    return path

if __name__ == "__main__":
    graph = {
        'S': ['C', 'A'],
        'A': ['S', 'C', 'B'],
        'C': ['S', 'A', 'B', 'G'],
        'B': ['A', 'C', 'G'],
        'G': ['C', 'B']
    }
```

```
h_manhattan = {
    'S': 12,          # |0-11| + |4-5| = 11 + 1 = 12
    'A': 13,          # |3-11| + |0-5| = 8 + 5 = 13
    'C': 8,           # |5-11| + |3-5| = 6 + 2 = 8
    'B': 6,           # |9-11| + |1-5| = 2 + 4 = 6
    'G': 0            # |11-11| + |5-5| = 0
}

final_path = hill_climbing(graph, h_manhattan, start='S', goal='G')
print("Final Traversed Path:", final_path)
```

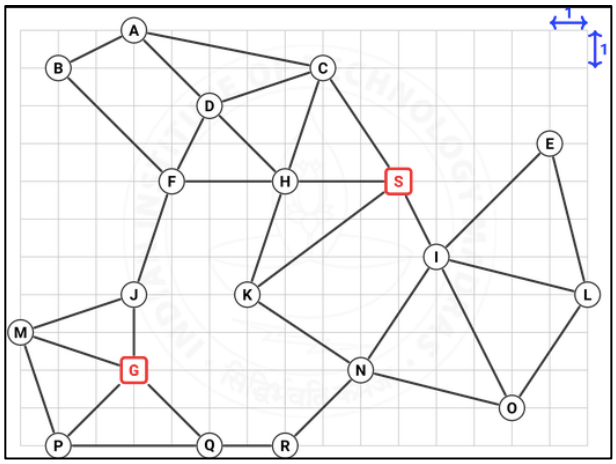
Code Explanation:



Output:

```
=== STARTING HILL CLIMBING SEARCH (MANHATTAN) ===
Current Node: S (h=12) --> Best Neighbor: C (h=8)
Current Node: C (h=8) --> Best Neighbor: G (h=0)
Success! Goal reached.
Final Traversed Path: ['S', 'C', 'G']
```

NOTE: Implementing the same hill climbing algorithm for a newer graph as given:

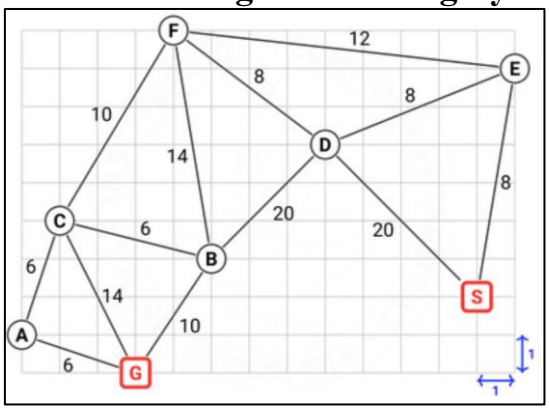


Output:

```
=== STARTING HILL CLIMBING SEARCH (MANHATTAN) ===
Current Node: S (h=12) --> Best Neighbor: K (h=5)
Current Node: K (h=5) --> Best Neighbor: N (h=6)
Local optimum reached. Halting search.
Failed to reach the goal.
Final Traversed Path: ['S', 'K']
```

EXPERIMENT – 5

5.1: To implement the A* Search Algorithm using Python.



Apparatus: Python, and heapq

Theory: The A* Search Algorithm is an informed search algorithm used to find the shortest or least-cost path between a starting node and a goal node. It combines the advantages of Uniform Cost Search and Greedy Best-First Search by considering both the actual cost required to reach a node and an estimated cost from that node to the goal.

A* evaluates each node using the following evaluation function:

f(n) = g(n) + h(n)

where:

- g(n) = actual cost of the path from the starting node to node n.
- h(n) = heuristic estimate of the cost from node n to the goal node.
- f(n) = estimated total cost of a path through node n.

The node having the lowest f(n) value is selected for exploration.

In this implementation, the graph is represented using a Python dictionary. Each node contains its neighboring nodes along with the corresponding edge costs. The coordinates of each node are also stored in a dictionary and are used to calculate heuristic values.

Heuristic Function: A heuristic function estimates the remaining cost from the current node to the goal. This program supports two heuristic methods:

1. **Manhattan Distance**: The Manhattan distance between two points is calculated as:
   h(n) = $|x - x_G| + |y - y_G|$
   where (x, y) represents the coordinates of the current node and (x_G, y_G) represents the coordinates of the goal node.
   In this program, the Manhattan distance is used by default.
2. **Euclidean Distance**: The Euclidean distance is calculated as:
   h(n) = $\sqrt{(x - x_G)^2 + (y - y_G)^2}$
   The program rounds the Euclidean distance to two decimal places.

Data Structures Used:

i. **Graph**: The graph is represented using a Python dictionary. Each node stores its neighboring nodes and the cost of reaching each neighbor.

ii. **Coordinates**: A dictionary stores the (x, y) coordinates of every node. These coordinates are used to calculate heuristic values.

iii. **Heuristic Dictionary**: A dictionary stores the calculated heuristic value h(n) for every node.

iv. **OPEN List**: A priority queue is used to store nodes that are discovered but have not yet been completely explored. Python's heapq module is used to implement this priority queue.

v. **Visited Set**: A Python set stores nodes that have already been explored to avoid unnecessarily processing them again.

vi. **g-score**: A dictionary stores the current lowest known cost from the starting node to every node.

vii. **Parent Dictionary**: The parent dictionary stores the previous node of every node in the currently selected path. It is used to reconstruct the final path after reaching the goal.

Working Principle: The A* algorithm begins by assigning a g value of 0 to the starting node and infinity to all other nodes. The starting node is inserted into the OPEN list with its corresponding f(n) value. At every step, the node with the lowest f(n) value is removed from the OPEN list and selected for exploration. For each neighboring node, the algorithm calculates a tentative path cost using: tentative_g(n) = g(current) + cost(current, n)
The heuristic value h(n) is obtained from the heuristic dictionary, and the corresponding evaluation value is calculated as: f(n) = tentative_g(n) + h(n)
If the newly calculated path provides a cheaper route to the neighboring node, its g value and parent are updated, and the node is inserted into the OPEN list. This process continues until the goal node is selected. Once the goal is reached, the parent dictionary is followed backwards from the goal to the starting node to reconstruct the complete path. The algorithm therefore considers both the cost already travelled and the estimated cost remaining, allowing it to efficiently search for a low-cost path.

Code:

```
import math
import heapq

def calculate_heuristics(coords: dict, goal:str = 'G', method:str ='manhattan') -> dict:
    gx, gy = coords[goal]
    h = {}

    for node, (x, y) in coords.items():
        if method == 'manhattan':
            h[node] = abs(x - gx) + abs(y - gy)
        elif method == 'euclidean':
            h[node] = round(math.sqrt((x - gx) ** 2 + (y - gy) ** 2), 2)
    return h

def a_star(graph: dict, h_manhattan: dict, start: str, goal: str):
    open = []
    visited = set()

    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    parent = {start: None}

    step = 0

    heapq.heappush(open, (g_score[start] + h_manhattan[start], start))
```

```
print("\n" + "=" * 65)
print("               A* SEARCH ALGORITHM")
print("=" * 65)

while open:
    step += 1

    print(f"\n{'—' * 65}")
    print(f"STEP {step}")
    print(f"{'—' * 65}")

    print("\nOPEN LIST:")
    for f, node in sorted(open):
        g = g_score[node]
        h = h_manhattan[node]
        print(
            f"  Node {node}: "
            f"g(n) = {g}, "
            f"h(n) = {h}, "
            f"f(n) = {f}"
        )

    current_f, current = heapq.heappop(open)

    current_g = g_score[current]
    current_h = h_manhattan[current]
    current_f = current_g + current_h

    print("\nSELECTED NODE:")
    print(
        f"  {current} "
        f"(g = {current_g}, "
        f"h = {current_h}, "
        f"f = {current_f})"
    )

    if current in visited:
        print("  Status: SKIPPED (already visited)")
        continue

    if current == goal:
        print("\nGOAL REACHED!")
        path = []
        node = current
        while node is not None:
            path.append(node)
            node = parent[node]

        path.reverse()
```

```
            print("\nFINAL PATH:")
            print(" -> ".join(path))
            print(f"TOTAL PATH COST: {g_score[goal]}")
            print("=" * 65)
            return

        visited.add(current)
        print("\nVISITED NODES:")
        print("  ", visited)

        print(f"\nEXPLORING NEIGHBORS OF {current}:")
        for neighbor, cost in graph[current].items():
            print(f"\n  Neighbor: {neighbor}")
            print(f"  Edge cost: {cost}")

            if neighbor in visited:
                print("  Status: SKIPPED (already visited)")
                continue

            tentative_g = g_score[current] + cost
            h = h_manhattan[neighbor]
            f = tentative_g + h

            print(f"  Tentative g(n): {tentative_g}")
            print(f"  Heuristic h(n): {h}")
            print(f"  Calculated f(n): {f}")

            if tentative_g < g_score[neighbor]:
                print("  Status: UPDATED (cheaper path found)")
                g_score[neighbor] = tentative_g
                parent[neighbor] = current
                heapq.heappush(open, (f, neighbor))
                print("  Added to OPEN list.")

            else:
                print("  Status: SKIPPED (no cheaper path)")

    print("\nNO PATH FOUND.")

if __name__ == "__main__":
    graph = {
        'A': {'C': 6 , 'G': 6},
        'B': {'C': 6, 'G': 10, 'D': 20, 'F': 14},
        'C': {'A': 6, 'B': 6, 'F': 10, 'G': 14},
        'D': {'B': 20, 'E': 8, 'F': 8, 'S': 20},
        'E': {'S': 8, 'F': 12, 'D': 8},
        'F': {'B': 14, 'C': 10, 'D': 8, 'E': 12},
        'G': {'A': 6, 'B': 10, 'C': 14},
        'S': {'E': 8, 'D': 20}
    }
```

```
coords = {
    'A': (0, 1), 'B': (5, 3), 'C': (1, 4), 'D': (8, 6), 'E': (13, 8), 'F': (4, 9), 'G': (3, 0), 'S': (12, 2),
}

h_manhattan = calculate_heuristics(coords)

print("Calculated Manhattan Distance Heuristics h(n):")
for node, val in sorted(h_manhattan.items()):
    print(f"  Node {node}: h({node}) = {val}")

a_star(graph, h_manhattan, 'S', 'G')
```

Output:

```
Calculated Manhattan Distance Heuristics h(n):
  Node A: h(A) = 4
  Node B: h(B) = 5
  Node C: h(C) = 6
  Node D: h(D) = 11
  Node E: h(E) = 18
  Node F: h(F) = 10
  Node G: h(G) = 0
  Node S: h(S) = 11

================================================================
                A* SEARCH ALGORITHM
================================================================
```

```
----------------------------------------------------------------
STEP 1
----------------------------------------------------------------

OPEN LIST:
  Node S: g(n) = 0, h(n) = 11, f(n) = 11

SELECTED NODE:
  S (g = 0, h = 11, f = 11)

VISITED NODES:
   {'S'}

EXPLORING NEIGHBORS OF S:

  Neighbor: E
  Edge cost: 8
  Tentative g(n): 8
  Heuristic h(n): 18
  Calculated f(n): 26
  Status: UPDATED (cheaper path found)
  Added to OPEN list.

  Neighbor: D
  Edge cost: 20
  Tentative g(n): 20
  Heuristic h(n): 11
  Calculated f(n): 31
  Status: UPDATED (cheaper path found)
  Added to OPEN list.
```

```
STEP 2

OPEN LIST:
  Node E: g(n) = 8, h(n) = 18, f(n) = 26
  Node D: g(n) = 20, h(n) = 11, f(n) = 31

SELECTED NODE:
  E (g = 8, h = 18, f = 26)

VISITED NODES:
   {'S', 'E'}

EXPLORING NEIGHBORS OF E:

  Neighbor: S
  Edge cost: 8
  Status: SKIPPED (already visited)

  Neighbor: F
  Edge cost: 12
  Tentative g(n): 20
  Heuristic h(n): 10
  Calculated f(n): 30
  Status: UPDATED (cheaper path found)
  Added to OPEN list.

  Neighbor: D
  Edge cost: 8
  Tentative g(n): 16
  Heuristic h(n): 11
  Calculated f(n): 27
  Status: UPDATED (cheaper path found)
  Added to OPEN list.
```

```
STEP 3

OPEN LIST:
  Node D: g(n) = 16, h(n) = 11, f(n) = 27
  Node F: g(n) = 20, h(n) = 10, f(n) = 30
  Node D: g(n) = 16, h(n) = 11, f(n) = 31

SELECTED NODE:
  D (g = 16, h = 11, f = 27)

VISITED NODES:
   {'S', 'D', 'E'}

EXPLORING NEIGHBORS OF D:

  Neighbor: B
  Edge cost: 20
  Tentative g(n): 36
  Heuristic h(n): 5
  Calculated f(n): 41
  Status: UPDATED (cheaper path found)
  Added to OPEN list.

  Neighbor: E
  Edge cost: 8
  Status: SKIPPED (already visited)

  Neighbor: F
  Edge cost: 8
  Tentative g(n): 24
  Heuristic h(n): 10
  Calculated f(n): 34
  Status: SKIPPED (no cheaper path)
```

```
  Neighbor: S
  Edge cost: 20
  Status: SKIPPED (already visited)

--------------------------------------------------
STEP 4
--------------------------------------------------

OPEN LIST:
  Node F: g(n) = 20, h(n) = 10, f(n) = 30
  Node D: g(n) = 16, h(n) = 11, f(n) = 31
  Node B: g(n) = 36, h(n) = 5, f(n) = 41

SELECTED NODE:
  F (g = 20, h = 10, f = 30)

VISITED NODES:
   {'S', 'D', 'F', 'E'}

EXPLORING NEIGHBORS OF F:

  Neighbor: B
  Edge cost: 14
  Tentative g(n): 34
  Heuristic h(n): 5
  Calculated f(n): 39
  Status: UPDATED (cheaper path found)
  Added to OPEN list.

  Neighbor: C
  Edge cost: 10
  Tentative g(n): 30
  Heuristic h(n): 6
  Calculated f(n): 36
  Status: UPDATED (cheaper path found)
  Added to OPEN list.

  Neighbor: D
  Edge cost: 8
  Status: SKIPPED (already visited)

  Neighbor: E
  Edge cost: 12
  Status: SKIPPED (already visited)

--------------------------------------------------
STEP 5
--------------------------------------------------

OPEN LIST:
  Node D: g(n) = 16, h(n) = 11, f(n) = 31
  Node C: g(n) = 30, h(n) = 6, f(n) = 36
  Node B: g(n) = 34, h(n) = 5, f(n) = 39
  Node B: g(n) = 34, h(n) = 5, f(n) = 41

SELECTED NODE:
  D (g = 16, h = 11, f = 27)
  Status: SKIPPED (already visited)

--------------------------------------------------
STEP 6
--------------------------------------------------

OPEN LIST:
  Node C: g(n) = 30, h(n) = 6, f(n) = 36
  Node B: g(n) = 34, h(n) = 5, f(n) = 39
  Node B: g(n) = 34, h(n) = 5, f(n) = 41

SELECTED NODE:
  C (g = 30, h = 6, f = 36)

VISITED NODES:
   {'S', 'D', 'C', 'E', 'F'}
```

```
EXPLORING NEIGHBORS OF C:

  Neighbor: A
  Edge cost: 6
  Tentative g(n): 36
  Heuristic h(n): 4
  Calculated f(n): 40
  Status: UPDATED (cheaper path found)
  Added to OPEN list.

  Neighbor: B
  Edge cost: 6
  Tentative g(n): 36
  Heuristic h(n): 5
  Calculated f(n): 41
  Status: SKIPPED (no cheaper path)

  Neighbor: F
  Edge cost: 10
  Status: SKIPPED (already visited)

  Neighbor: G
  Edge cost: 14
  Tentative g(n): 44
  Heuristic h(n): 0
  Calculated f(n): 44
  Status: UPDATED (cheaper path found)
  Added to OPEN list.
```

```
--------------------------------------------------
STEP 7
--------------------------------------------------

OPEN LIST:
  Node B: g(n) = 34, h(n) = 5, f(n) = 39
  Node A: g(n) = 36, h(n) = 4, f(n) = 40
  Node B: g(n) = 34, h(n) = 5, f(n) = 41
  Node G: g(n) = 44, h(n) = 0, f(n) = 44

SELECTED NODE:
  B (g = 34, h = 5, f = 39)

VISITED NODES:
   {'S', 'B', 'D', 'C', 'E', 'F'}

EXPLORING NEIGHBORS OF B:

  Neighbor: C
  Edge cost: 6
  Status: SKIPPED (already visited)

  Neighbor: G
  Edge cost: 10
  Tentative g(n): 44
  Heuristic h(n): 0
  Calculated f(n): 44
  Status: SKIPPED (no cheaper path)

  Neighbor: D
  Edge cost: 20
  Status: SKIPPED (already visited)

  Neighbor: F
  Edge cost: 14
  Status: SKIPPED (already visited)
```

```
------------------------------------------------------------
STEP 8
------------------------------------------------------------

OPEN LIST:
  Node A: g(n) = 36, h(n) = 4, f(n) = 40
  Node B: g(n) = 34, h(n) = 5, f(n) = 41
  Node G: g(n) = 44, h(n) = 0, f(n) = 44

SELECTED NODE:
  A (g = 36, h = 4, f = 40)

VISITED NODES:
   {'S', 'B', 'D', 'C', 'E', 'A', 'F'}

EXPLORING NEIGHBORS OF A:

  Neighbor: C
  Edge cost: 6
  Status: SKIPPED (already visited)

  Neighbor: G
  Edge cost: 6
  Tentative g(n): 42
  Heuristic h(n): 0
  Calculated f(n): 42
  Status: UPDATED (cheaper path found)
  Added to OPEN list.
```

```
------------------------------------------------------------
STEP 9
------------------------------------------------------------

OPEN LIST:
  Node B: g(n) = 34, h(n) = 5, f(n) = 41
  Node G: g(n) = 42, h(n) = 0, f(n) = 42
  Node G: g(n) = 42, h(n) = 0, f(n) = 44

SELECTED NODE:
  B (g = 34, h = 5, f = 39)
  Status: SKIPPED (already visited)

------------------------------------------------------------
STEP 10
------------------------------------------------------------

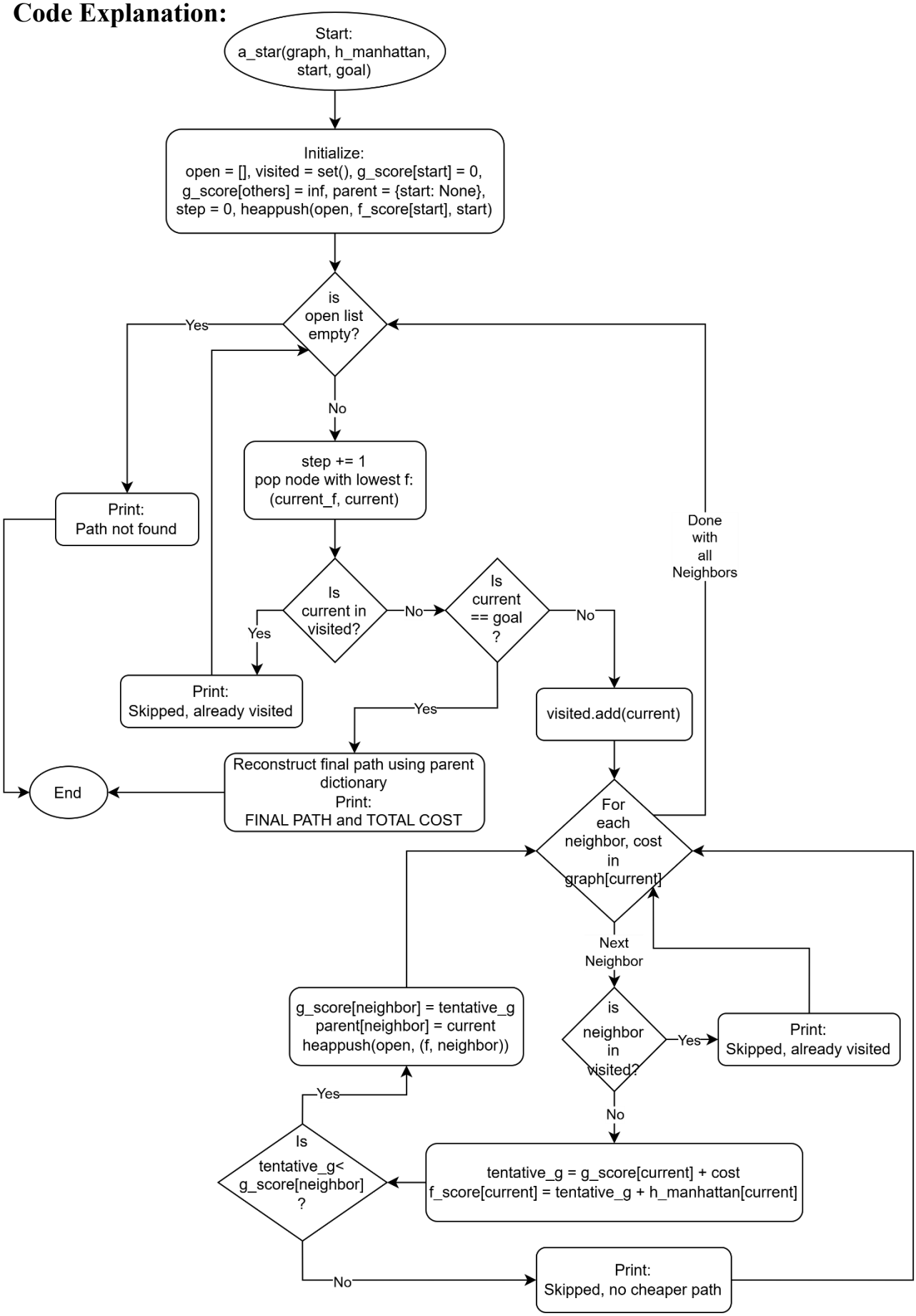
OPEN LIST:
  Node G: g(n) = 42, h(n) = 0, f(n) = 42
  Node G: g(n) = 42, h(n) = 0, f(n) = 44

SELECTED NODE:
  G (g = 42, h = 0, f = 42)

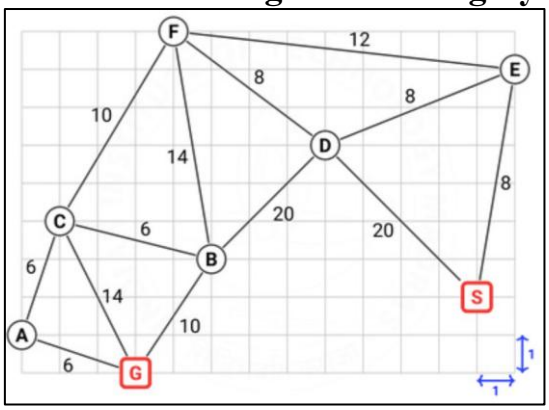
GOAL REACHED!

FINAL PATH:
S -> E -> F -> C -> A -> G
TOTAL PATH COST: 42
============================================================
```

Code Explanation:



5.2: To implement the wA* Search Algorithm using Python script.



Apparatus: Python, and heapq

Theory: The Weighted A* (WA*) Search Algorithm is an informed search algorithm used to find a low-cost path between a starting node and a goal node. It is a modified version of the standard A* Search Algorithm that gives additional importance to the heuristic estimate while selecting nodes for exploration.

Standard A* evaluates each node using:

$$f(n) = g(n) + h(n)$$

Weighted A* modifies this evaluation function as:

$$f(n) = g(n) + w \times h(n)$$

where:

- g(n) = actual cost of the path from the starting node to node n.
- h(n) = heuristic estimate of the cost from node n to the goal node.
- w = weight assigned to the heuristic function, generally w > 1.
- f(n) = weighted estimated total cost of reaching the goal through node n.

A larger value of w gives greater importance to the heuristic. Therefore, Weighted A* tends to explore nodes that appear closer to the goal and can reduce the amount of search required compared with standard A*. However, depending on the heuristic and weight used, the resulting path may not always be the exact optimal path.

In this implementation, the graph is represented using a Python dictionary. Each node contains its neighboring nodes along with the corresponding edge costs. The coordinates of each node are stored separately and are used to calculate the heuristic values.

Working Principle: The Weighted A* algorithm begins by assigning a g value of 0 to the starting node and infinity to all other nodes. A heuristic value h(n) is calculated for every node based on its distance from the goal. The starting node is inserted into the OPEN list with its weighted evaluation value: $f(n) = g(n) + w \times h(n)$

At every step, the node having the lowest f(n) value is removed from the OPEN list and selected for exploration. For each neighboring node, the algorithm calculates a tentative path cost using: $tentative_g(n) = g(current) + cost(current, n)$

The heuristic value of the neighboring node is then obtained and the weighted evaluation value is calculated as: $f(n) = tentative_g(n) + w \times h(n)$

If the newly calculated path provides a cheaper route to the neighboring node, its g value and parent are updated, and the node is inserted into the OPEN list.

The process continues until the goal node is selected. Once the goal is reached, the parent dictionary is followed backwards from the goal to the starting node to reconstruct the complete path.

The main difference between standard A* and Weighted A* is that Weighted A* gives greater importance to the heuristic estimate. Therefore, increasing the weight generally makes the search more goal-directed and can reduce the number of nodes explored, at the possible cost of path optimality.

Code:

```
import math
import heapq

def calculate_heuristics(coords: dict, goal: str = 'G', method: str ='manhattan') -> dict:
    gx, gy = coords[goal]
    h = {}

    for node, (x, y) in coords.items():
        if method == 'manhattan':
            h[node] = abs(x - gx) + abs(y - gy)
        elif method == 'euclidean':
            h[node] = round(math.sqrt((x - gx) ** 2 + (y - gy) ** 2), 2)
    return h

def weighted_a_star(graph: dict, h_manhattan: dict, start: str, goal: str, weight: float):
    open = []
    visited = set()

    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    parent = {start: None}

    initial_f = g_score[start] + weight * h_manhattan[start]
    heapq.heappush(open, (initial_f, start))

    while open:
        current_f, current = heapq.heappop(open)

        if current in visited:
            continue

        if current == goal:
            path = []
            node = current

            while node is not None:
                path.append(node)
                node = parent[node]

            path.reverse()
            return path, g_score[goal]

        visited.add(current)
```

```
        for neighbor, cost in graph[current].items():
            if neighbor in visited:
                continue

            tentative_g = g_score[current] + cost
            h = h_manhattan[neighbor]
            f = tentative_g + weight * h

            if tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                parent[neighbor] = current

                heapq.heappush(open, (f, neighbor))

    return list(visited), None

if __name__ == "__main__":
    graph = {
        'A': {'C': 6 , 'G': 6},
        'B': {'C': 6, 'G': 10, 'D': 20, 'F': 14},
        'C': {'A': 6, 'B': 6, 'F': 10, 'G': 14},
        'D': {'B': 20, 'E': 8, 'F': 8, 'S': 20},
        'E': {'S': 8, 'F': 12, 'D': 8},
        'F': {'B': 14, 'C': 10, 'D': 8, 'E': 12},
        'G': {'A': 6, 'B': 10, 'C': 14},
        'S': {'E': 8, 'D': 20}
    }

    coords = {
        'A': (0, 1),
        'B': (5, 3),
        'C': (1, 4),
        'D': (8, 6),
        'E': (13, 8),
        'F': (4, 9),
        'G': (3, 0),
        'S': (12, 2),
    }

    h_manhattan = calculate_heuristics(coords)

    print("\nMANHATTAN DISTANCE HEURISTICS")
    print("-" * 70)

    for node, value in sorted(h_manhattan.items()):
        print(f"h({node}) = {value}")

    weights = [1, 2, 3, 5]
```

```
print("TEST CASES")
print("-" * 70)

for weight in weights:
    path, cost = weighted_a_star(graph, h_manhattan, 'S', 'G', weight)

    print(f"\nWeight = {weight}")
    print("-" * 70)

    if cost is not None:
        print(f"Path       : {' -> '.join(path)}")
        print(f"Total Cost : {cost}")
    else:
        print(f"Visited    : {' -> '.join(path)}")
        print("Total Cost : Goal not reached")
```

Output:

```
MANHATTAN DISTANCE HEURISTICS
----------------------------------------------------------------------
h(A) = 4
h(B) = 5
h(C) = 6
h(D) = 11
h(E) = 18
h(F) = 10
h(G) = 0
h(S) = 11
TEST CASES
----------------------------------------------------------------------

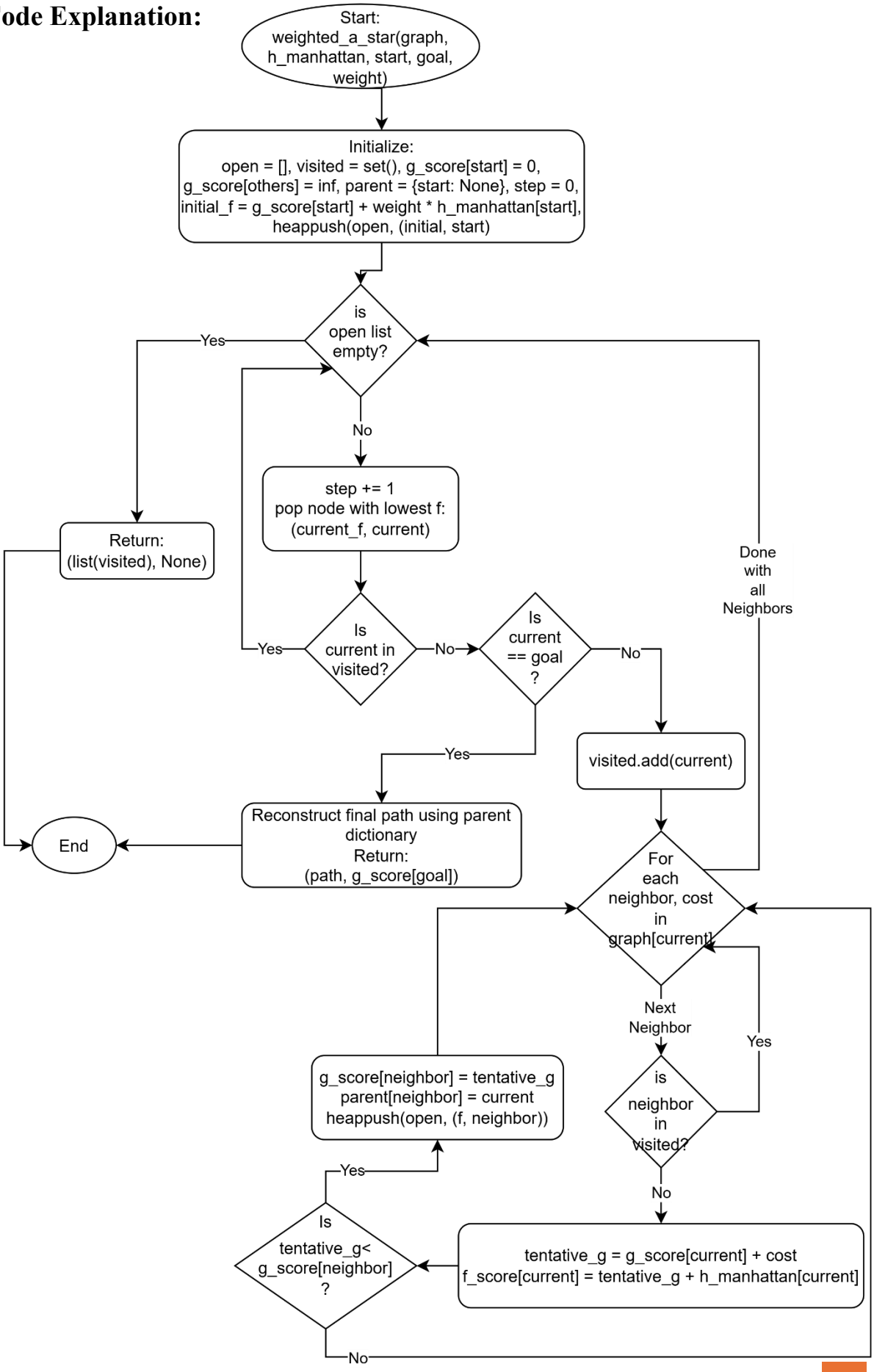
Weight = 1
----------------------------------------------------------------------
Path       : S -> E -> F -> C -> A -> G
Total Cost : 42

Weight = 2
----------------------------------------------------------------------
Path       : S -> E -> F -> C -> A -> G
Total Cost : 42

Weight = 3
----------------------------------------------------------------------
Path       : S -> D -> B -> G
Total Cost : 50

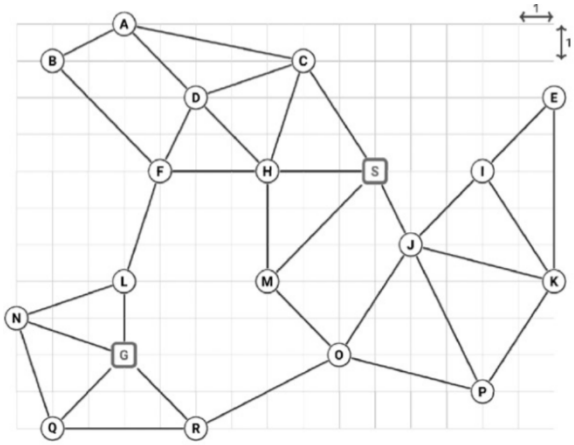
Weight = 5
----------------------------------------------------------------------
Path       : S -> D -> B -> G
Total Cost : 50
```

Code Explanation:



EXPERIMENT – 6

6.1: To implement DB-DFS Search Algorithm for given unweighted graph using Python.



Apparatus: Python

Theory: Depth-Bounded Depth-First Search (DB-DFS) is a search algorithm used in Artificial Intelligence to find a goal node in a graph by performing Depth-First Search up to a specified depth limit. It is a modification of Depth-First Search (DFS) in which the search is not allowed to go beyond a predefined depth bound. This prevents the algorithm from searching indefinitely in a very deep or infinite search space.

A DB-DFS problem consists of the following components:

1. Initial State: The node from which the search starts. In this program, the initial state is S.
2. Goal State: The node that the algorithm tries to find. In this program, the goal state is G.
3. Graph: The graph represents the relationships between different nodes. It is stored using a Python dictionary called MOVE_GEN, where each node contains a list of its neighbouring nodes.
4. Depth Bound: The maximum depth up to which the search is allowed. It is provided by the user during program execution.
5. OPEN: A list that stores the nodes that are yet to be explored. The nodes are stored along with their parent and depth in the form: (node, parent, depth)
6. CLOSED: A set that stores the nodes that have already been visited. It prevents the same node from being expanded repeatedly.
7. Parent: A dictionary that stores the parent of each node. It is used to reconstruct the path from the starting node to the goal node.

Data Structures Used:

i. MOVE_GEN: A Python dictionary is used to represent the graph. Each key represents a node and its value contains the neighbouring nodes.

ii. OPEN: A Python list is used to store nodes that are waiting to be explored. It contains the node, its parent, and its depth.

iii. CLOSED: A Python set is used to store already visited nodes.

iv. Parent: A Python dictionary stores the parent of each node and helps in finding the final path.

v. Path: A Python list is used to store path from the starting node S to the goal node G.

Functions Used:

1. goal_test(node): This function checks whether the current node is the goal node. It returns True when the current node is G.

2. move_gen(node): This function returns the neighbouring nodes of the given node from the MOVE_GEN dictionary.

3. occurs_in(node, node_pairs): This function checks whether a node is already present in OPEN. This prevents duplicate nodes from being added to the search.

4. reconstruct_path(parent, goal): This function reconstructs the path from the goal node back to the starting node using the parent dictionary. The path is then reversed to obtain the correct order from S to G.

5. db_dfs(start, depth_bound): This is the main DB-DFS function. It maintains OPEN and CLOSED, performs the goal test, expands nodes only when their depth is less than the depth bound, and adds new nodes to the front of OPEN to perform DFS.

Working Principle: The DB-DFS process starts with the initial node S, which is inserted into OPEN with depth 0. The algorithm removes a node from OPEN and checks whether it is the goal node G. If it is the goal, the path is reconstructed using the parent dictionary and returned. If the node is not the goal, it is added to CLOSED. The algorithm then checks the depth of the node. If the depth is less than the specified depth bound, its neighbouring nodes are generated using move_gen(). Nodes that are already present in OPEN or CLOSED are ignored. The remaining nodes are added to OPEN along with their parent and new depth. The new nodes are inserted at the front of OPEN. Therefore, the most recently generated nodes are explored first, giving the search Depth-First Search behaviour.

The process continues until:

- the goal node G is found, or
- OPEN becomes empty, meaning no solution exists within the given depth bound.

Code:

```
MOVE_GEN = {
   'S': ['C', 'H', 'J', 'M'], 'A': ['B', 'C', 'D'], 'B': ['A', 'F'], 'C': ['A', 'D', 'H', 'S'],
   'D': ['A', 'C', 'F', 'H'], 'E': ['I', 'K'], 'F': ['B', 'D', 'H', 'L'], 'G': ['L', 'N', 'Q', 'R'],
   'H': ['C', 'D', 'F', 'M', 'S'], 'I': ['E', 'J', 'K'], 'J': ['I', 'K', 'O', 'P', 'S'],
   'K': ['E', 'I', 'J', 'P'], 'L': ['F', 'G', 'N'], 'M': ['H', 'O', 'S'], 'N': ['G', 'L', 'Q'],
   'O': ['J', 'M', 'P', 'R'], 'P': ['J', 'K', 'O'], 'Q': ['G', 'N', 'R'], 'R': ['G', 'O', 'Q']
}

def goal_test(node):
   return node == 'G'

def move_gen(node):
   return MOVE_GEN.get(node, [])

def occurs_in(node, node_pairs):
   for pair in node_pairs:
      if pair[0] == node:
         return True
   return False

def reconstruct_path(parent, goal):
```

```
    path = []
    node = goal

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()
    return path

def db_dfs(start, depth_bound):
    OPEN = [(start, None, 0)]
    CLOSED = set()
    parent = {start: None}

    while OPEN:
        node, node_parent, depth = OPEN.pop(0)

        if goal_test(node):
            path = reconstruct_path(parent, node)
            return path, CLOSED, depth

        CLOSED.add(node)

        if depth < depth_bound:
            children = move_gen(node)
            new_nodes = []

            for child in children:
                if (not occurs_in(child, OPEN) and child not in CLOSED):
                    new_nodes.append(child)
                    parent[child] = node

            new_pairs = []
            for child in new_nodes:
                new_pairs.append((child, node, depth + 1))

            OPEN = new_pairs + OPEN

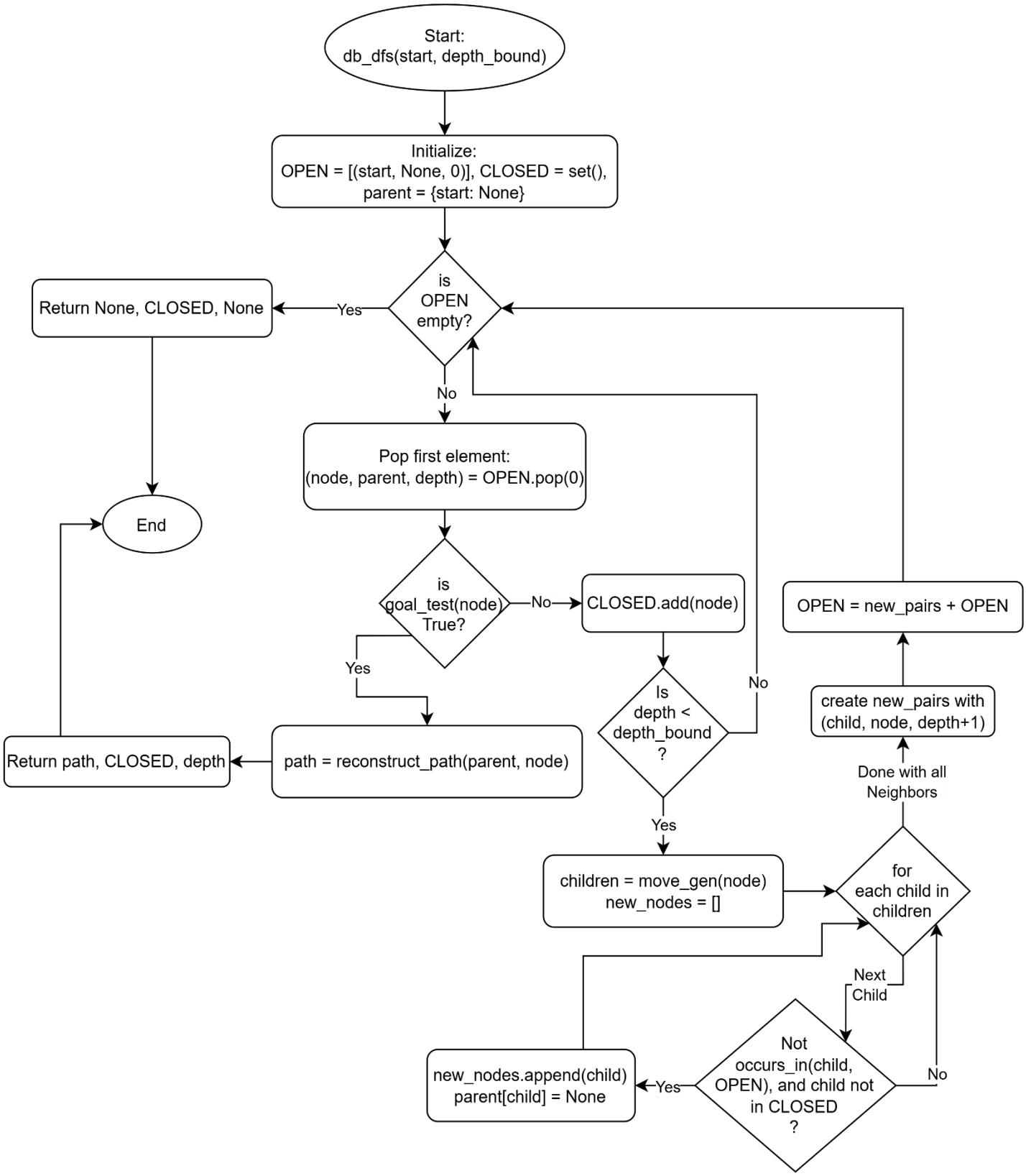
    return None, CLOSED, None

start = 'S'
depth_bound = int(input("Enter depth bound: "))
path, closed, depth = db_dfs(start, depth_bound)
print("\nCLOSED:", list(closed))

if path is not None:
    print("Goal found!")
    print("Path:", " -> ".join(path))
    print("Depth:", depth)
```

```
else:
    print("Goal not found within depth bound.")
```

Code Explanation:



Output:

```
Enter depth bound: 3

CLOSED: ['D', 'J', 'P', 'A', 'I', 'S', 'R', 'H', 'O', 'K', 'F', 'B', 'M', 'E', 'C']
Goal not found within depth bound.
```

```
Enter depth bound: 4

CLOSED: ['H', 'F', 'A', 'R', 'S', 'I', 'K', 'J', 'D', 'C', 'E', 'O', 'B']
Goal found!
Path: S -> J -> O -> R -> G
Depth: 4
```

6.2: To implement DFID Search Algorithm for the same graph as in 6.1 using Python.